

MultiTASC: A Multi-Tenancy-Aware Scheduler for Cascaded DNN Inference at the Consumer Edge

Sokratis Nikolaidis[†], Stylianos I. Venieris[‡], and Iakovos S. Venieris[†]

[†]National Technical University of Athens, Athens, Greece, [‡]Samsung AI Center, Cambridge, UK

Email: sokratisnikolaidis@mail.ntua.gr, s.venieris@samsung.com, venieris@cs.ece.ntua.gr

Abstract—Cascade systems comprise a two-model sequence, with a lightweight model processing all samples and a heavier, higher-accuracy model conditionally refining harder samples to improve accuracy. By placing the light model on the device side and the heavy model on a server, model cascades constitute a widely used distributed inference approach. With the rapid expansion of intelligent indoor environments, such as smart homes, the new setting of Multi-Device Cascade is emerging where multiple and diverse devices are to simultaneously use a shared heavy model on the same server, typically located within or close to the consumer environment. This work presents MultiTASC, a multi-tenancy-aware scheduler that adaptively controls the forwarding decision functions of the devices in order to maximize the system throughput, while sustaining high accuracy and low latency. By explicitly considering device heterogeneity, our scheduler improves the latency service-level objective (SLO) satisfaction rate by 20-25 percentage points (pp) over state-of-the-art cascade methods in highly heterogeneous setups, while serving over 40 devices, showcasing its scalability.

I. INTRODUCTION

In recent years, there has been significant progress in the field of on-device execution of deep learning (DL) inference tasks [1]. At the same time, with the rapid expansion of indoor intelligent environments [2], such as smart homes and offices, DL is poised to enable new use-cases by expanding to a greater variety of smart devices, such as IoT cameras and AI speakers. Nevertheless, due to their form-factor and energy-efficiency constraints, most of these devices lie on the low end of the computational spectrum. In contrast to modern high-end smartphones, which host powerful processors and accelerators (GPUs, NPUs) [3], low-end devices are not able to deploy state-of-the-art deep neural networks (DNNs), resorting to lightweight, but lower-accuracy models.

Given that offloading data to the cloud for inference can incur significant costs in terms of bandwidth, latency and privacy, an alternative scheme is emerging that places the server inside or closer to the consumer environment in the form of a dedicated AI hub that assists the surrounding devices [2]. In this context, a prominent deployment approach are the cascade architectures [4]–[9]. Cascade architectures make use of the fact that not all samples are of the same difficulty and choose to process only the more challenging cases with a powerful model deployed on the server, while letting easier samples, which usually form the majority of the data, to be processed on-device with a light model. A lot of research has been conducted on such architectures, mainly focusing on the forwarding decision criterion and selection of model pairs, progressing the potency of the scheme.

Despite the progress, the majority of these works have solely focused on the setting where the server is used by a *single* device at any given time. Such an assumption no longer holds in upcoming intelligent environments where multiple devices execute DL inference tasks simultaneously under the support of the same AI hub [10]. This gives rise to the new setting of *Multi-Device Cascade*, where multiple devices use the same model on a shared edge-based server. Such a system needs to be scalable in terms of number of devices, balancing fast response time and high accuracy across them. In this context, status-quo approaches, which treat each model cascade independently, would either lead to brute-forcing inference requests through the server’s resources, resulting in the system being overwhelmed, or force all devices to fallback to on-device execution, negating any accuracy benefits. Therefore, there is an emerging need for novel methods that explicitly target the challenges of a Multi-Device Cascade.

In this work, we propose MultiTASC, a multi-tenancy-aware scheduler that allows the Multi-Device Cascade architecture to adapt to dynamic conditions by reconfiguring on-the-fly the forwarding criterion of the cascades, thus controlling the server’s inference request rate at run time. To sustain smooth operation under device heterogeneity, we further introduce a heterogeneity-aware prioritization scheme that selectively adapts each device’s operation depending on its capabilities. Overall, MultiTASC sustains high responsiveness, throughput and accuracy while the number of assisted devices scales in both homogeneous and heterogeneous device ecosystems. The key contributions of this paper are the following:

- A system model of the Multi-Device Cascade architecture. By expanding the cascade architecture to accommodate multiple devices, our parametrization exposes the tunable parameters and enables system designers to systematically investigate its trade-offs.
- A multi-tenancy-aware scheduler optimized for the Multi-Device Cascade architecture. The proposed scheduler aims to maximize throughput and accuracy while satisfying a latency constraint. This is accomplished through the adaptive manipulation of the forwarding decision functions of the devices that control the inference request flow to the server. By introducing a new metric, Capacity, our scheduler estimates the maximum amount of samples that can be processed on the server within a given latency constraint and utilizes it to dynamically reconfigure the forwarding decision functions on the assisted devices.

MultiTASC: 用于消费者边缘级联 DNN 推理的多租户感知调度程序

Sokratis Nikolaidis[†]、Stylianos I. Venieris[‡] 和 Iakovos S. Venieris[†] 希腊雅典
国立雅典技术大学, [‡]英国剑桥三星人工智能中心 电子邮件: sokratisnikolaidis@mail.
ntua.gr、s.venieris@samsung.com、venieris@cs.ece.ntua.gr

Abstract—级联系统由两个模型序列组成, 一个轻量级模型处理所有样本, 一个较重、精度更高的模型有条件地精炼较难的样本以提高精度。模型级联将轻模型放在设备端, 重模型放在服务器端, 构成了广泛使用的分布式推理方法。随着智能家居等智能室内环境的快速扩展, 多设备级联的新设置正在出现, 其中多个不同的设备将同时使用同一服务器上的共享重型模型, 通常位于消费者环境内或附近。这项工作提出了 MultiTASC, 这是一种多租户感知调度程序, 它自适应地控制设备的转发决策功能, 以最大限度地提高系统吞吐量, 同时保持高精度和低延迟。通过明确考虑设备异构性, 我们的调度程序将延迟服务级别目标 (SLO) 满意度比高度异构设置中最先进的级联方法提高了 20-25 个百分点 (pp), 同时为 40 多个设备提供服务, 展示了其可扩展性。

一、简介

近年来, 深度学习 (DL) 推理任务的设备端执行领域取得了重大进展[1]。与此同时, 随着智能家居和办公室等室内智能环境[2]的快速扩展, 深度学习有望通过扩展到更多种智能设备 (例如物联网摄像头和人工智能扬声器) 来实现新的用例。然而, 由于其外形尺寸和能效限制, 这些设备中的大多数都处于计算范围的低端。与配备强大处理器和加速器 (GPU、NPU) [3] 的现代高端智能手机相比, 低端设备无法部署最先进的深度神经网络 (DNN), 只能采用轻量级但精度较低的模型。

鉴于将数据卸载到云端进行推理可能会在带宽、延迟和隐私方面产生巨大成本, 因此正在出现一种替代方案, 即以辅助周围设备的专用 AI 中心的形式将服务器放置在消费者环境内部或更靠近消费者环境 [2]。在这种情况下, 一种突出的部署方法是级联架构[4]-[9]。级联架构利用了这样一个事实, 即并非所有样本都具有相同的难度, 并选择使用部署在服务器上的强大模型仅处理更具挑战性的案例, 同时让通常构成大部分数据的更简单的样本使用轻模型在设备上进行处理。人们对这种架构进行了大量的研究, 主要集中在转发决策标准和模型对的选择上, 提高了该方案的效力。

尽管取得了进展, 但这些工作中的大多数都只关注 *single* 设备在任何给定时间使用服务器的设置。这种假设在即将到来的智能环境中不再成立, 其中多个设备在同一人工智能中心的支持下同时执行深度学习推理任务[10]。这就产生了 *Multi-Device Cascade* 的新设置, 其中多个设备在共享的基于边缘的服务器上使用相同的模型。这样的系统需要在设备数量方面具有可扩展性, 并在设备之间平衡快速响应时间和高精度。在这种情况下, 独立处理每个模型级联的现状方法要么会导致通过服务器资源进行暴力推理请求, 导致系统不堪重负, 要么迫使所有设备回退到设备上执行, 从而抵消任何准确性优势。因此, 迫切需要明确针对多设备级联挑战的新方法。

在这项工作中, 我们提出了 MultiTASC, 这是一种多租户感知调度程序, 它允许多设备级联架构通过动态重新配置级联的转发标准来适应动态条件, 从而在运行时控制服务器的推理请求率。为了在设备异构性下保持平稳运行, 我们进一步引入了一种异构感知优先级方案, 该方案根据其功能有选择地调整每个设备的操作。总体而言, MultiTASC 保持高响应能力、吞吐量和准确性, 同时辅助设备的数量在同构和异构设备生态系统中扩展。本文的主要贡献如下:

- 多设备级联架构的系统模型。通过扩展级联架构以适应多个设备, 我们的参数化公开了可调参数, 并使系统设计人员能够系统地研究其权衡。
- 针对多设备级联架构优化的多租户感知调度程序。所提出的调度程序旨在最大化吞吐量和准确性, 同时满足延迟约束。这是通过控制到服务器的推理请求流的设备的转发决策功能的自适应操纵来实现的。通过引入新的指标“容量”, 我们的调度程序可以估计在给定的延迟限制内服务器上可以处理的最大样本量, 并利用它动态地重新配置辅助设备上的转发决策功能。

II. BACKGROUND & RELATED WORK

On-Device DNN Inference. In recent years, the on-device deployment of DL models has rapidly gained ground [1]. Although on-device training remains a distant possibility, inference has been achieved on computationally constrained devices through the utilization of various methods, such as lightweight model design [11], quantization [12], pruning [13], knowledge distillation [14], and optimized scheduling [15]. Still, despite the maturity of these techniques, upcoming intelligent spaces, such as smart homes and offices, are often populated with small-form factor, resource-constrained devices (e.g. smart cameras, AI speakers), which lack the processing power to support high-accuracy, computationally intensive models. This fact has motivated the development of distributed collaborative inference approaches.

Distributed Collaborative Inference. Distributed inference systems employ a server to assist mobile and embedded devices in performing DNN inference tasks. This approach has given rise to two main schemes: *offloading* and *cascading*. Offloading [16]–[18] leverages the server’s resources to alleviate part of the computational load on the device by splitting the DNN model into two parts; the first part is executed on the device, with the intermediate results forwarded to the server to proceed with the execution of the second part.

In the cascade scheme, samples are fed into a two-model sequence of DNNs, with progressively increasing complexity and accuracy. Once the input is processed by a model, its output is evaluated by a forwarding decision function, which determines whether the inference ends using the current result or proceeds to the next model. Several works have been conducted on this area. [4] proposes to analyze the difference between the best and the second best softmax result in order to determine which inputs can be processed only by the light model and which require the heavy model. [5] introduces a trainable forwarding criterion by training a neural head on the light model’s feature extractor. [6] explores the idea of using more than two DNN models and compares different decision metrics. Finally, [7] and [8] propose solutions for deploying cascades under tight energy constraints.

Multi-Device Cascades. Previous works on cascades [4]–[8] have focused on an isolated setting, where a single device has exclusive access to a server. Nonetheless, with an increased rate of AI-enabled applications deployed in indoor environments, this assumption is unrealistic; instead, multiple devices will require support from the same server at any given time. Such a setting requires principled investigation, since a brute-force deployment, where devices run independently of each other, would lead to either an overloaded server, and in turn to long response times, or significant drop in accuracy by falling back to local execution. The Multi-Device Cascade setting remains unexplored and is the focus of this work.

III. MULTI-DEVICE CASCADE OF CLASSIFIERS

Fig. 1 presents the system architecture of a Multi-Device Cascade, where IoT devices are running DL inference tasks. All devices perform the same task, e.g. object detection, but

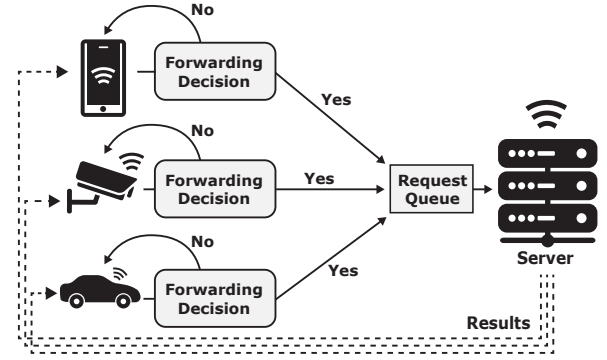


Fig. 1: System architecture of Multi-Device Cascade.

may host different models. The output predictions of each sample are passed on to a *Forwarding Decision* function that determines whether the DL model of the device is confident about its output. Depending on the decision, either the result remains as is, or the sample is forwarded to a server to be processed by a more accurate model. The forwarded samples from all devices are put into a *Request Queue* from where they are drawn to form the input of the server-hosted model; as such, the server-side model is *shared* among all devices. Finally, the results produced on the server are sent back to their corresponding devices.

Single-Device Cascade. Let $x \in \mathcal{X}$ be the input of a DL inference task performed on an IoT device and $y \in \{1, \dots, K\}$ the classification label produced by the model, where K is the number of classes. By using a decision function, $d(\cdot)$, on the output of the light device model, we can decide whether the result is satisfactory ($d(\cdot)=0$) or we should forward the sample to the heavier model for further processing ($d(\cdot)=1$). Denoting the classification function of the light model by $f_l : \mathcal{X} \rightarrow [0, 1]^K$ that yields the softmax output vector of the model whose maximum value is the predicted class, and the classification of the heavy model by $f_h : \mathcal{X} \rightarrow [0, 1]^K$, we formally define a collaborative cascade system as:

$$casc_{f_l, f_h, d}(x) = \begin{cases} f_l(x) & \text{if } d(f_l(x)) = 0 \\ f_h(x) & \text{if } d(f_l(x)) = 1 \end{cases} \quad (1)$$

Multi-Device Cascade. To capture Multi-Device Cascade architectures (Fig. 1), we extend the single-device cascade system modeling as follows. Let $\mathcal{D}$ be the set of devices that use the server as a collaborator. Then, the Multi-Device Cascade system is defined as:

$$casc_{f_l^i, f_h, d^i}(x^i) = \begin{cases} f_l^i(x^i) & \text{if } d^i(f_l^i(x)) = 0 \quad \forall i \in \{1, \dots, |\mathcal{D}|\} \\ f_h(x^i) & \text{if } d^i(f_l^i(x)) = 1 \end{cases}$$

where $x^i \in \mathcal{X}^i$ is a sample processed by the i -th device, f_l^i the classification function of the DL model deployed on the i -th device, f_h the shared heavy model on the server, and $d^i(f_l^i(x))$ the forwarding decision function of the i -th device.

Congestion Problem. When a single device is using the server as the collaborator, it has exclusive access to the server’s computational resources and hence the response time is minimized. With the proliferation of IoT devices, such a

二.背景及相关工作

设备上 DNN 推理。近年来，深度学习模型在设备上的部署迅速普及[1]。尽管设备上训练仍然是一个遥远的可能性，但通过利用各种方法，例如轻量级模型设计[11]、量化[12]、剪枝[13]、知识蒸馏[14]和优化调度[15]，已经在计算受限的设备上实现了推理。尽管如此，尽管这些技术已经成熟，但即将到来的智能空间，例如智能家居和办公室，通常会配备体积小、资源有限的设备（e.g.智能相机、人工智能扬声器），这些设备缺乏支持高精度、计算密集型模型的处理能力。这一事实推动了分布式协作推理方法的发展。

分布式协作推理。分布式推理系统使用服务器来协助移动和嵌入式设备执行 DNN 推理任务。这种方法产生了两个主要方案：*offloading*和*cascading*。卸载[16]-[18]通过将 DNN 模型分成两部分，利用服务器的资源来减轻设备上的部分计算负载；第一部分在设备上执行，中间结果转发到服务器以继续执行第二部分。

在级联方案中，样本被输入 DNN 的双模型序列，其复杂性和准确性逐渐增加。一旦模型处理输入，其输出将由转发决策函数评估，该函数确定推理是使用当前结果结束还是继续到下一个模型。在这一领域已经开展了多项工作。[4]建议分析最佳和次佳softmax结果之间的差异，以确定哪些输入只能由轻模型处理，哪些输入需要重模型。[5]通过在光模型的特征提取器上训练神经头来引入可训练的转发标准。[6]探索了使用两个以上 DNN 模型的想法并比较了不同的决策指标。最后，[7]和[8]提出了在严格的能源约束下部署级联的解决方案。

多设备级联。先前关于级联的工作[4]-[8]主要关注隔离设置，其中单个设备对服务器具有独占访问权限。尽管如此，随着在室内环境中部署人工智能应用程序的比例不断增加，这种假设是不现实的。相反，多个设备在任何给定时间都需要同一服务器的支持。这样的设置需要原则性的调查，因为设备彼此独立运行的强力部署会导致服务器过载，进而导致响应时间过长，或者由于回退到本地执行而导致准确性显著下降。多设备级联设置仍未被探索，是本工作的重点。

三. 多设备级联的分类器

图 1 展示了多设备级联的系统架构，其中物联网设备正在运行深度学习推理任务。所有设备执行相同的任务，e.g. 对象检测，但是

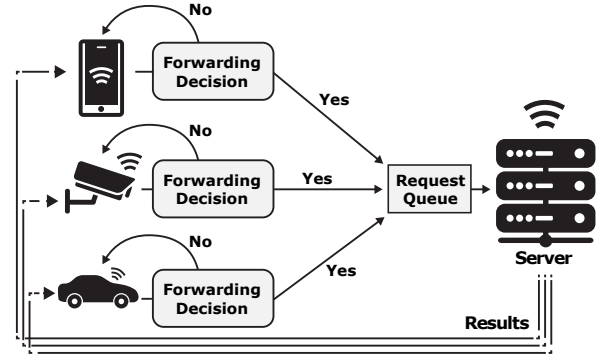


图1: 多设备级联系统架构。

可能承载不同的模型。每个样本的输出预测都会传递给 *Forwarding Decision* 函数，该函数确定设备的深度学习模型对其输出是否有信心。根据决策，结果要么保持原样，要么将样本转发到服务器以由更准确的模型进行处理。来自所有设备的转发样本被放入 *Request Queue* 中，从那里提取它们以形成服务器托管模型的输入；因此，所有设备中服务器端模型为 *shared*。最后，服务器产生的结果被发送回相应的设备。

单设备级联。令 $x \in \mathcal{X}$ 为在 IoT 设备上执行的深度学习推理任务的输入， $y \in \{1, \dots, K\}$ 为模型生成的分类标签，其中 K 是类别数。通过对轻设备模型的输出使用决策函数 $d(\cdot)$ ，我们可以决定结果是否令人满意 ($d(\cdot)=0$)，或者我们应该将样本转发到较重的模型进行进一步处理 ($d(\cdot)=1$)。用 $f_l: \mathcal{X} \rightarrow [0, 1]^K$ 表示轻模型的分函数，产生最大值为预测类别的模型的 softmax 输出向量，用 $f_h: \mathcal{X} \rightarrow [0, 1]^K$ 表示重模型的分函数，我们正式定义协作级联系统为：

$$casc_{f_l, f_h, d}(x) = \begin{cases} f_l(x) & \text{if } d(f_l(x)) = 0 \\ f_h(x) & \text{if } d(f_l(x)) = 1 \end{cases} \quad (1)$$

多设备级联。为了捕获多设备级联架构（图 1），我们按如下方式扩展单设备级联系统建模。令 $\mathcal{D}$ 为使用服务器作为协作者的设备集。那么，多设备级联系统定义为：

$$casc_{f_l^i, f_h, d^i}(x^i) = \begin{cases} f_l^i(x^i) & \text{if } d^i(f_l^i(x^i)) = 0 \quad \forall i \in \{1, \dots, |\mathcal{D}|\} \\ f_h(x^i) & \text{if } d^i(f_l^i(x^i)) = 1 \end{cases}$$

其中， $x^i \in \mathcal{X}^i$ 是第 i 个设备处理的样本， f_l^i 是部署在第 i 个设备上的 DL 模型的分函数， f_h 是服务器上的共享重模型， $d^i(f_l^i(x^i))$ 是第 i 个设备的转发决策函数。

拥塞问题。当单个设备使用服务器作为协作者时，它可以独占访问服务器的计算资源，从而最大限度地缩短响应时间。随着物联网设备的激增，诸如

server should be utilized by many devices at the same time, to amortize its cost and provide maximum utility. Nonetheless, depending on the conditions, if the arrival rate of incoming requests exceeds the attainable processing throughput of the server, the server will be overloaded and the requests will experience severe waiting times in the request queue.

Given the number of devices $|\mathcal{D}|$, we express the arrival rate of requests to the server as $AR_{\text{server}} = \sum_{i=1}^{|\mathcal{D}|} p_{\text{casc}}^i / t_{\text{inf}}^i$ where t_{inf}^i is the average inference latency of a sample on the i -th device and p_{casc}^i is the probability of a sample giving $d^i(f_l^i(x^i)) = 1$. Given the attainable throughput T_{server} of the server, we distinguish between three different states:

- $AR_{\text{server}} < T_{\text{server}}$: the server's processing rate is larger than the arrival rate, resulting in the server being under-utilized. A larger number of difficult samples could be sent to the server to achieve higher accuracy.
- $AR_{\text{server}} = T_{\text{server}}$: equilibrium is attained. Requests are processed upon arrival without accumulating and the server's processing power is fully utilized.
- $AR_{\text{server}} > T_{\text{server}}$: the requests arrive faster than the server can process. If this state lasts, the request queue will be large, resulting in unwanted latency.

Since the probability p_{casc}^i of the forwarding decision function is not static, but changes during inference, such an architecture could benefit by dynamically adapting its state depending on the current conditions. Since t_{inf}^i and T_{server} are fixed based on the device and server-side processors, we opt to manipulate p_{casc}^i by changing the parameters of $d^i(f_l^i(x^i))$ in order to introduce adaptability to the system.

Problem Optimization. We formulate the aforementioned problem as a multi-objective optimization problem, aiming to maximize accuracy and throughput subject to a latency service-level objective (SLO). The next section describes our proposed scheduler that tackles this problem.

IV. PROPOSED SOLUTION

To combat the accumulation of requests or underutilization of server resources, we propose MultiTASC, a multi-tenancy-aware scheduler that dynamically adapts the forwarding decision functions on assisted devices in order to control the arrival rate of samples. Fig. 2 depicts its internal design. The scheduler monitors the state of the request queue, communicates with the assisted devices and tunes the flow of incoming requests based on the current conditions. To this end, we introduce four techniques: *i*) reconfigurable forwarding decision functions, *ii*) the Capacity metric on the server, *iii*) fractional update, and *iv*) device heterogeneity-aware prioritization for effectively communicating updates to the devices.

A. Reconfigurable Forwarding Decision Function

Research effort has been invested into quantifying the prediction confidence of DNNs, leading to several approaches [6], [19]. In this work, we adopt the Best-versus-Second-Best (BvSB) metric [20], using the difference between the two highest values of the softmax outputs of the model as

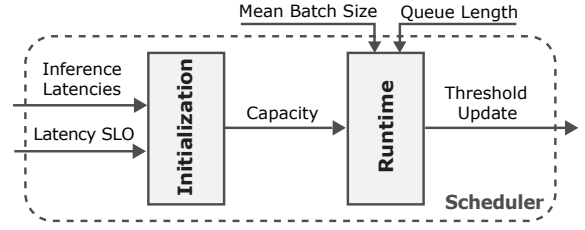


Fig. 2: MultiTASC's internal architecture.

$BvSB|_{f(x)} = P_1 - P_2$, where P_1 and P_2 are the maximum and second maximum values, respectively, in the output softmax vector of classifier $f(x)$. Other metrics, such as top-1 softmax or entropy, can be used with minimal modifications, potentially leading to different latency-accuracy trade-offs.

The vast majority of existing cascade systems opts to select a specific threshold at design time, which then remains fixed upon deployment. In this work, to accommodate the adaptability needs of our target system, we adopt an alternative scheme where the decision function can be *dynamically reconfigured*. The decision function $d^i(\cdot)$ is thus defined as:

$$d^i(f_l^i(x)) = \begin{cases} 0 & \text{if } BvSB|_{f_l^i(x)} \geq c^{i,t} \\ 1 & \text{if } BvSB|_{f_l^i(x)} < c^{i,t} \end{cases} \quad (2)$$

where $c^{i,t}$ is the decision threshold of device i at time t . The per-device decision thresholds are exposed to our server-residing scheduler, which adapts them at run time.

B. The Capacity Metric

To fully take advantage of the server's computational resources and boost throughput, it is important to use batching, *i.e.* processing multiple samples at the same time. To avoid the latency that would arise from waiting for the request queue to reach a specific batch size, we employ dynamic batching [21]. With dynamic batching, we use the maximum batch size that is feasible with the current request queue length. Available batch sizes are $\mathcal{B} = \{1, 2, 4, 8, 16, 32, 64\}$. Due to diminishing returns, in some cases we use a lower maximum batch size, *e.g.* with EfficientNetB3 a batch size of 16 provides higher throughput and lower latency than a batch size of 32 and above.

To calculate the amount of samples that can be processed by the server within a given latency constraint, we introduce the Capacity metric. Based on the inference latency of each batch size and a given latency SLO, the scheduler calculates the maximum amount of samples that can be classified without latency violations. We call that amount Capacity.

To obtain the value of the server's Capacity, we cast the problem as an unbounded variation of the Knapsack problem, where the batch size throughput is analogous to the value/weight ratio. Since we know that the larger the batch size the higher the throughput (up to the point where the server's processing power is saturated), we introduce a greedy

服务器应该同时被许多设备使用，以分摊其成本并提供最大的效用。尽管如此，根据情况，如果传入请求的到达率超过服务器可达到的处理吞吐量，服务器将过载，并且请求将在请求队列中经历严重的等待时间。

给定设备数量 $|\mathcal{D}|$ ，我们将请求到达服务器的速率表示为 $AR_{\text{server}} = \sum_{i=1}^{|\mathcal{D}|} p_{\text{casc}}^i / t_{\text{inf}}^i$ ，其中 t_{inf}^i 是第 i 个设备上样本的平均推理延迟， p_{casc}^i 是样本给出 $d^i(f_l^i(x^i)) = 1$ 的概率。给定服务器的可达到吞吐量 T_{server} ，我们区分三种不同的状态：

- $AR_{\text{server}} < T_{\text{server}}$: 服务器的处理速率大于到达速率，导致服务器利用率不足。可以将更多的困难样本发送到服务器以实现更高的准确性。
- $AR_{\text{server}} = T_{\text{server}}$: 达到平衡。请求到达后立即处理，不会累积，充分利用服务器的处理能力。
- $AR_{\text{server}} > T_{\text{server}}$: 请求到达的速度比服务器处理的速度快。如果这种状态持续下去，请求队列将会很大，从而导致不必要的延迟。

由于转发决策函数的概率 p_{casc}^i 不是静态的，而是在推理过程中发生变化，因此这种架构可以通过根据当前条件动态调整其状态来受益。由于 t_{inf}^i 和 T_{server} 是根据设备和服务器端处理器固定的，因此我们选择通过更改 $d^i(f_l^i(x^i))$ 的参数来操作 p_{casc}^i ，以引入系统的适应性。

问题优化。我们将上述问题表述为多目标优化问题，旨在根据延迟服务级别目标（SLO）最大化准确性和吞吐量。下一节将介绍我们提出的解决此问题的调度程序。

四. 建议的解决方案

为了解决请求累积或服务器资源利用不足的问题，我们提出了 MultiTASC，这是一种多租户感知调度程序，可以动态调整辅助设备上的转发决策功能，以控制样本的到达率。图2描述了其内部设计。调度程序监视请求队列的状态，与辅助设备通信并根据当前条件调整传入请求的流。为此，我们引入了四种技术：i)可重新配置的转发决策函数、ii)服务器上的容量指标、iii)部分更新和iv)设备异构感知优先级，用于有效地将更新传达给设备。

A. Reconfigurable Forwarding Decision Function

研究工作已投入到量化 DNN 的预测置信度上，从而产生了多种方法 [6]、[19]。在这项工作中，我们采用最佳与第二最佳（BvSB）度量 [20]，使用模型的 softmax 输出的两个最高值之间的差作为

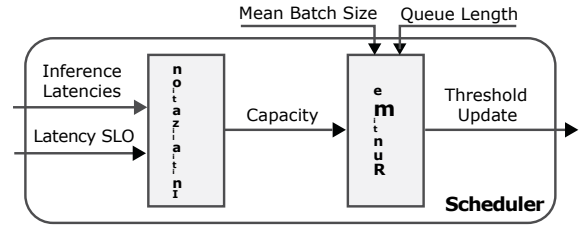


图2: MultiTASC的内部架构。

$BvSB \Big|_{f(x)} = P_1 - P_2$ ，其中 P_1 和 P_2 分别是分类器 $f(x)$ 的输出 softmax 向量中的最大和第二最大值。其他指标，例如 to p-1 softmax 或熵，可以在进行最小修改的情况下使用，可能会导致不同的延迟精度权衡。

绝大多数现有级联系统选择在设计时选择特定阈值，然后在部署时保持固定。在这项工作中，为了适应目标系统的适应性需求，我们采用了另一种方案，其中决策函数可以是 *dynamically reconfigured*。因此，决策函数 $d^i(\cdot)$ 定义为：

$$d^i(f_l^i(x)) = \begin{cases} 0 & \text{if } BvSB \Big|_{f_l^i(x)} \geq c^{i,t} \\ 1 & \text{if } BvSB \Big|_{f_l^i(x)} < c^{i,t} \end{cases} \quad (2)$$

其中 $c^{i,t}$ 是设备 i 在时间 t 的决策阈值。每个设备的决策阈值都会暴露给我们驻留在服务器的调度程序，该调度程序会在运行时对其进行调整。

B. The Capacity Metric

为了充分利用服务器的计算资源并提高吞吐量，使用批处理、同时处理多个样本非常重要。为了避免等待请求队列达到特定批量大小而产生的延迟，我们采用动态批处理 [21]。通过动态批处理，我们使用当前请求队列长度可行的最大批处理大小。可用的批量大小为 $\mathcal{B} = \{1, 2, 4, 8, 16, 32, 64\}$ 。由于收益递减，在某些情况下，我们使用较低的最大批处理大小，e.g., EfficientNetB3 的批处理大小为 16，与批处理大小为 32 及以上相比，可提供更高的吞吐量和更低的延迟。

为了计算服务器在给定延迟限制内可以处理的样本量，我们引入了容量指标。根据每个批量大小的推理延迟和给定的延迟 SLO，调度程序计算在不违反延迟的情况下可以分类的最大样本量。我们称这个量为容量。

为了获得服务器容量的值，我们将问题转化为背包问题的无界变体，其中批量大小吞吐量类似于价值/重量比。由于我们知道批量大小越大，吞吐量越高（直到服务器处理能力饱和），因此我们引入了贪婪算法

TABLE I: Evaluated DNN Models

Model	Location	Device	Clock Rate	Accuracy	Latency	FLOPs	#Params
MobileNetV2	Low-end	Sony Xperia C5	1.69 GHz	71.85%	31 ms	0.6 B	3.5 M
EfficientNetLite0	Mid-tier	Samsung A71	2.20 GHz	75.02%	43 ms	0.8 B	4.7 M
EfficientNetB0	High-end	Samsung S20 FE	2.73 GHz	77.04%	33 ms	0.8 B	5.3 M
InceptionV3	Server	Tesla T4 GPU	585 MHz	78.29%	15 ms	11.4 B	23.8 M
EfficientNetB3	Server	Tesla T4 GPU	585 MHz	81.49%	25 ms	3.7 B	12.2 M

* See Table 1 in [1] for the detailed resource characteristics of the target mobile phones.

algorithm that adds the largest batch size as many times as possible. Let $\mathcal{B}$ be the pool of batch sizes, then:

$$C = \max_n \sum_{j=1}^{|\mathcal{B}|} b_j n_j, \quad \text{s.t.} \quad \sum_{j=1}^{|\mathcal{B}|} L_{\text{inf}}^{b_j} n_j \leq L^{\text{SLO}}, \quad n_j \geq 0 \quad (3)$$

where C is the Capacity metric, b_j is the j -th batch size, n_j is the amount of times the batch size is used, $L_{\text{inf}}^{b_j}$ is the inference latency of the j -th batch size and L^{SLO} is the latency SLO.

C. Fractional Update

Changing the thresholds of all devices at once could lead to unwanted, sudden oscillations of the system. To achieve a smoother operation of the system and give enough time for each adaptation step to affect the execution, MultiTASC introduces the *fractional update* technique. With fractional update, MultiTASC updates the thresholds of only a certain percentage, denoted by P , of the total number of devices for each update. Thresholds are updated as dictated by Eq. (4). We set the invocation rate of the scheduler to be once every 2 seconds, allowing for update results to affect the system.

$$TC = \begin{cases} -M & \text{if } \bar{b} > \alpha \cdot C \quad \text{and} \quad QL > \alpha \cdot C \\ +M & \text{if } \bar{b} \leq \beta \cdot C \quad \text{and} \quad QL \leq \beta \cdot C \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

where M is the margin by which the threshold of the chosen devices changes, $\bar{b}$ the average of the last L batch sizes used for inference on the server, C the Capacity metric and QL the request queue length. Capacity is weighted using the parameters α and β . If requests accumulate beyond a certain limit despite the dynamic threshold updates, all thresholds are set to 0 until the server is decongested.

D. Device Heterogeneity-Aware Prioritization

MultiTASC explicitly considers device heterogeneity in order to maximize both the average accuracy and the system throughput. Our heterogeneity-aware prioritization strategy updates the threshold values using Eq. (4), but expands the pipeline by selecting which type of devices receive the update. The key insight of our strategy is that when it comes to decreasing the thresholds, MultiTASC prioritizes high-end and mid-tier devices since they host larger models and can maintain higher accuracy even with low thresholds. In contrast, when increasing thresholds, our scheduler prioritizes low-end devices, since they are the ones benefiting the most from higher thresholds due to hosting lighter models.

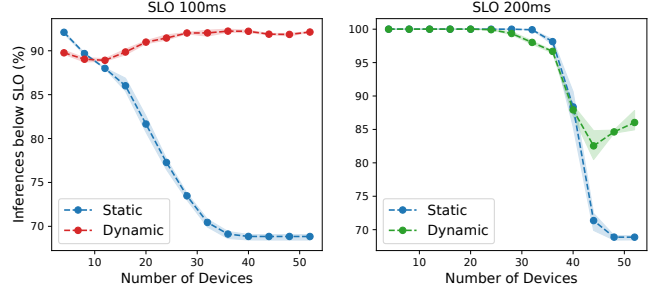


Fig. 3: SLO satisfaction rate for EfficientNetLite0-InceptionV3 pair.

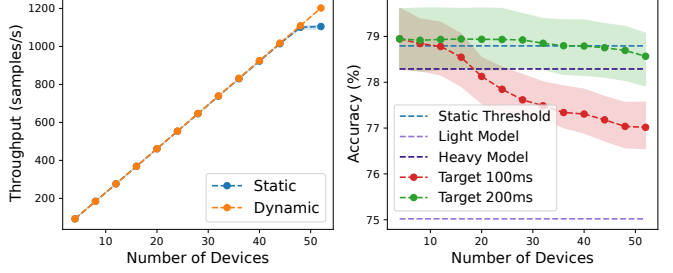


Fig. 4: Throughput and accuracy for EfficientNetLite0-InceptionV3 pair.

V. EVALUATION

A. Experimental Setup

To evaluate the performance of MultiTASC, we built a prototype on top of TensorFlow 2.9.1 targeting an edge server and three tiers of mobile devices. The edge server hosts an NVIDIA Tesla T4 GPU, Intel(R) Xeon(R) 2.30GHz CPU and 12GB of RAM. For the clients, we target three smartphones of increasing processing capabilities, namely: Sony Xperia C5 Ultra, Samsung A71 and Samsung S20 FE, representing low-, mid- and high-end clients, respectively. To assess our system across various settings, we conduct simulation-based experiments, varying the target latency SLO and the number of client devices. We measure the average inference time across 200 runs on the target devices for the evaluated models. We follow the same approach for the server-side models across different batch sizes. All on-device measurements were performed using TensorFlow Lite and targeting the CPU of the respective mobile device. For the device-server communication, we employ the AMQP protocol, following the widely used practice for communication between IoT devices.

Models & Datasets. We target the task of 1k-class image classification. Concretely, we use the ImageNet dataset and its 50k-images validation set in our experiments. Table I shows the evaluated models. We obtain the ImageNet-pretrained models as provided by TensorFlow Hub. On the client side, to emulate the common approach where models are selected based on the capabilities of each device, we deploy MobileNetV2, EfficientNetLite0 and EfficientNetB0 on low-, mid- and high-end devices, respectively. On the server side, we chose InceptionV3 and EfficientNetB3 as representative models that are computationally heavy, but provide high accuracy.

表 I: 评估的 DNN 模型

Model	Location	Device	Clock Rate	Accuracy	Latency	FLOPs	#Params
MobileNetV2	Low-end	Sony Xperia C5	1.69 GHz	71.85%	31 ms	0.6 B	3.5 M
EfficientNetLite0	Mid-tier	Samsung A71	2.20 GHz	75.02%	43 ms	0.8 B	4.7 M
EfficientNetB0	High-end	Samsung S20 FE	2.73 GHz	77.04%	33 ms	0.8 B	5.3 M
InceptionV3	Server	Tesla T4 GPU	585 MHz	78.29%	15 ms	11.4 B	23.8 M
EfficientNetB3	Server	Tesla T4 GPU	585 MHz	81.49%	25 ms	3.7 B	12.2 M

* 目标手机的详细资源特征参见[1]中的表1。

尽可能多次添加最大批量大小的算法。令 $\mathcal{B}$ 为批量大小池，然后：

$$C = \max_n \sum_{j=1}^{|\mathcal{B}|} b_j n_j, \quad \text{s.t.} \quad \sum_{j=1}^{|\mathcal{B}|} L_{\text{inf}}^{b_j} n_j \leq L^{\text{SLO}}, n_j \geq 0 \quad (3)$$

其中， C 是容量指标， b_j 是第 j 个批次大小， n_j 是使用批次大小的次数， $L_{\text{inf}}^{b_j}$ 是第 j 个批次大小的推理延迟， L^{SLO} 是延迟 SLO。

C. Fractional Update

立即更改所有设备的阈值可能会导致系统出现不必要的突然振荡。为了实现系统更平稳的运行，并为每个适应步骤提供足够的时间来影响执行，MultiTASC 引入了 *fractional update* 技术。通过部分更新，MultiTASC 仅更新每次更新的设备总数的一定百分比（用 P 表示）的阈值。阈值按照等式更新。（4）。我们将调度程序的调用频率设置为每 2 秒一次，允许更新结果影响系统。

$$TC = \begin{cases} -M & \text{if } \bar{b} > \alpha \cdot C \text{ and } QL > \alpha \cdot C \\ +M & \text{if } \bar{b} \leq \beta \cdot C \text{ and } QL \leq \beta \cdot C \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

其中， M 是所选设备阈值变化的幅度， $\bar{b}$ 是用于服务器上推理的最后 L 批次大小的平均值， C 是容量指标， QL 是请求队列长度。使用参数 α 和 β 对容量进行加权。如果尽管动态阈值更新，请求累积超过特定限制，则所有阈值都将设置为 0，直到服务器解除拥塞。

D. Device Heterogeneity-Aware Prioritization

MultiTASC 明确考虑设备异构性，以便最大限度地提高平均精度和系统吞吐量。我们的异质性感知优先级策略使用等式更新阈值。（4），但通过选择接收更新的设备类型来扩展管道。我们策略的关键见解是，在降低阈值时，MultiTASC 优先考虑高端和中端设备，因为它们承载更大的模型，并且即使在较低阈值的情况下也可以保持更高的精度。相反，当增加阈值时，我们的调度程序会优先考虑低端设备，因为它们是由于托管较轻的模型而从较高阈值中受益最多的设备。

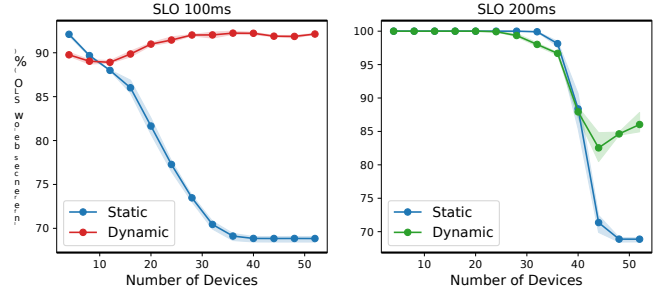


图 3: EfficientNetLite0-InceptionV3 对的 SLO 满意度。

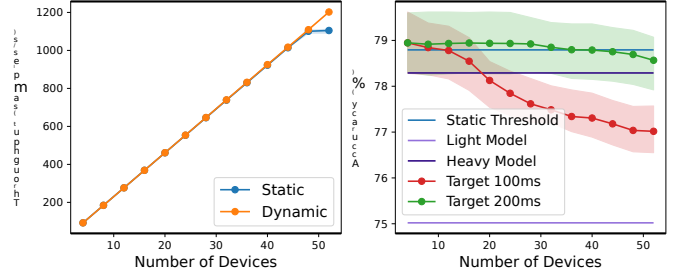


图 4: EfficientNetLite0-InceptionV3 对的吞吐量和准确性。

五、评价

A. Experimental Setup

为了评估 MultiTASC 的性能，我们在 TensorFlow 2.9.1 之上构建了一个原型，针对边缘服务器和三层移动设备。边缘服务器配备 NVIDIA Tesla T4 GPU、Intel(R) Xeon(R) 2.30GHz CPU 和 12GB RAM。对于客户而言，我们瞄准了三款处理能力不断增强的智能手机，即：索尼Xperia C5 Ultra、三星A71和三星S20 FE，分别代表低端、中端和高端客户。为了跨各种设置评估我们的系统，我们进行了基于模拟的实验，改变了目标延迟 SLO 和客户端设备的数量。我们测量了评估模型在目标设备上运行 200 次的平均推理时间。我们对不同批量大小的服务器端模型遵循相同的方法。所有设备上测量均使用 TensorFlow Lite 并针对相应移动设备的 CPU 进行。对于设备-服务器通信，我们采用 AMQP 协议，遵循物联网设备之间广泛使用的通信实践。

模型和数据集。我们的目标是 1k 类图像分类的任务。具体来说，我们在实验中使用 ImageNet 数据集及其 50k 图像验证集。表一显示了评估的模型。我们获得 TensorFlow Hub 提供的 ImageNet 预训练模型。在客户端，为了模拟根据每个设备的功能选择模型的常见方法，我们分别在低端、中端和高端设备上部署 MobileNetV2、EfficientNetLite0 和 EfficientNetB0。在服务器端，我们选择 InceptionV3 和 EfficientNetB3 作为代表模型，它们计算量大，但精度高。

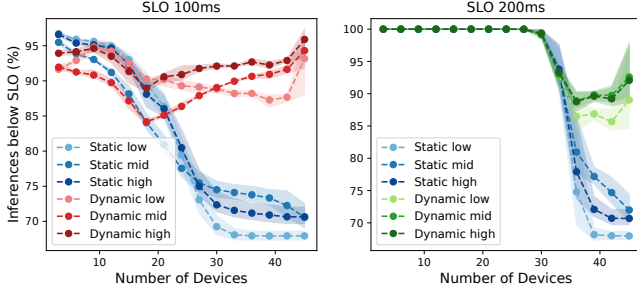


Fig. 5: SLO satisfaction rate for InceptionV3 on the server.

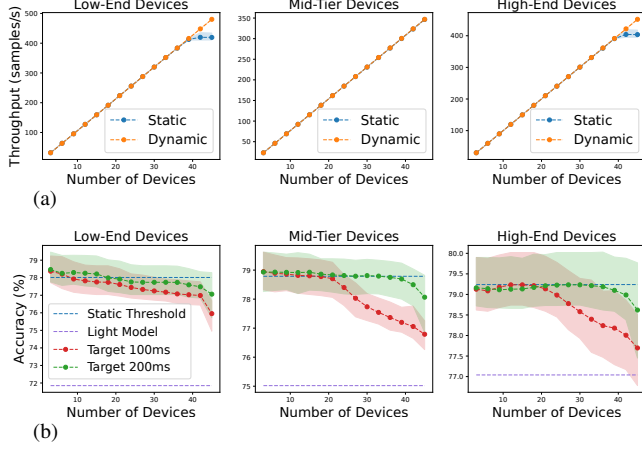


Fig. 6: (a) Throughput and (b) accuracy for InceptionV3 on the server.

Evaluation Settings. We focus on two types of device ecosystems: *i)* homogeneous, which comprises devices of equal processing capabilities that host the *same* local model; and *ii)* heterogeneous, which comprises devices of diverse processing capabilities, with each device hosting a model sized to its tier. In the homogeneous scenario, all devices were mid-tier, *i.e.* A71 phones, and ran EfficientNetLite0 with an average on-device inference latency of 43 ms. In the heterogeneous scenario, all three tiers of devices were deployed in equal percentage. In both cases, the dataset of each device consisted of 5,000 randomly selected samples from the last 40,000 images of ImageNet’s validation set. Three different seeds were used and the average is reported. The metrics used for the evaluation are: the system throughput, the average accuracy across devices, latency SLO satisfaction rate for 100- and 200-ms SLOs, and scalability in terms of number of devices.

Baseline. As a baseline, we use a scheduler with statically selected thresholds that remain fixed at run time. To choose the static threshold, we use the first 10,000 images of ImageNet’s validation set as our calibration set and evaluate all cascade model pairs in terms of accuracy and forwarding probability. As such, we tune the threshold so that approximately 30% of samples are forwarded to the heavy model, providing a balanced accuracy-latency trade-off. In cases where that threshold yielded an accuracy loss of more than 1 pp compared to the highest achievable cascade accuracy, we used the lowest threshold that satisfied the 1 pp limit. This baseline is equivalent to a set of state-of-the-art cascades [5], [6], [9].

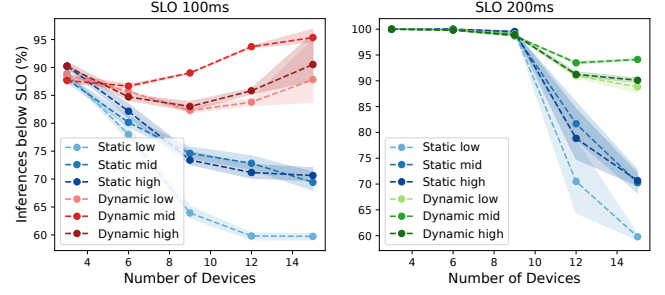


Fig. 7: SLO satisfaction rate for EfficientNetB3.

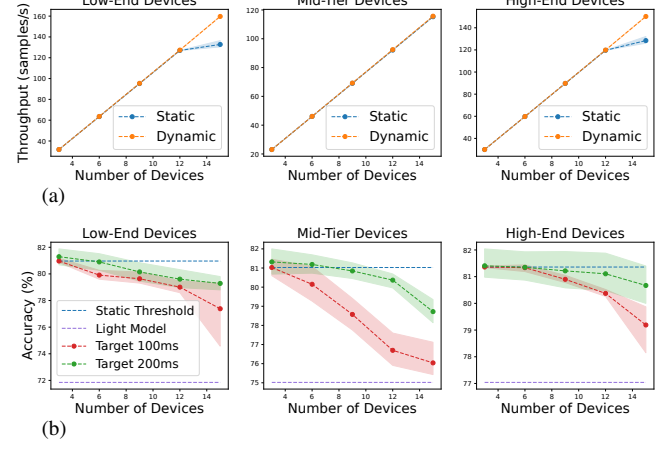


Fig. 8: (a) Throughput and (b) accuracy for EfficientNetB3 on the server.

B. Evaluation of MultiTASC’s Performance

Here, we assess the performance of our scheduler across both device ecosystems. We optimized MultiTASC’s parameters P , M , L , α and β using grid search and set them equal to 20%, 0.05, 5, 0.83 and 0.125, respectively.

Homogeneous Scenario. Fig. 3 shows the SLO satisfaction rate comparison for the pair EfficientNetLite0-InceptionV3. For a small amount of devices, MultiTASC keeps the satisfaction rate close to the baseline. As the baseline starts failing at 20 and 40 devices for SLOs of 100 and 200 ms, respectively, MultiTASC gradually reconfigures the thresholds of the forwarding decision functions and manages to keep satisfaction rate high, gaining 15-20 pp across the SLOs. In Fig. 4, we can see that after 45 devices the baseline’s throughput is starting to plateau, due to the queue not being served quickly enough. The users of the devices would experience this plateau as excessive response time. In contrast, with MultiTASC, the aggregate throughput continues to increase linearly with a growing number of devices, indicating that the requests are not allowed to accumulate and more devices can be served.

With respect to accuracy, for a small number of devices, MultiTASC slightly increases accuracy. This happens because our approach recognizes that the server is being underutilized and increases the thresholds accordingly. As the number of devices becomes larger, accuracy is traded off to achieve lower latency and sustain higher throughput, but still stays within 2 pp of the baseline. Additionally, we observe that the accuracy achieved by the cascade is in most cases higher than

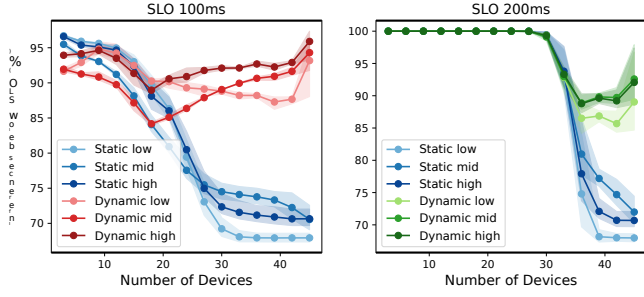


图 5: 服务器上 InceptionV3 的 SLO 满意度。

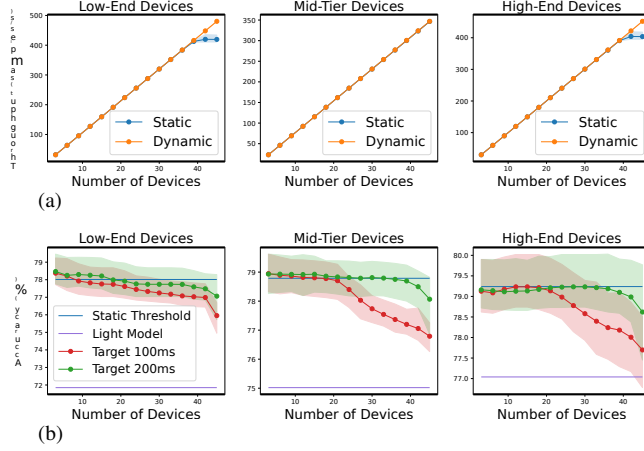


图 6: 服务器上 InceptionV3 的 (a) 吞吐量和 (b) 准确性。

评估设置。我们关注两种类型的设备生态系统: *i)* 同构, 由具有相同处理能力且托管 *same* 本地模型的设备组成; *ii)* 异构, 由具有不同处理能力的设备组成, 每个设备托管一个与其层大小相适应的模型。在同质场景中, 所有设备都是中层 *i.e.* A71 手机, 并运行 EfficientNetLite0, 平均设备上推理延迟为 43 毫秒。异构场景下, 三层设备部署比例相等。在这两种情况下, 每个设备的数据集都包含从 ImageNet 验证集的最后 40,000 张图像中随机选择的 5,000 个样本。使用三种不同的种子并报告平均值。用于评估的指标包括: 系统吞吐量、跨设备的平均准确度、100 毫秒和 200 毫秒 SLO 的延迟 SLO 满意度以及设备数量方面的可扩展性。

基线。作为基线, 我们使用具有静态选择阈值的调度程序, 这些阈值在运行时保持固定。为了选择静态阈值, 我们使用 ImageNet 验证集的前 10,000 张图像作为校准集, 并根据准确性和转发概率评估所有级联模型对。因此, 我们调整阈值, 以便将大约 30% 的样本转发到重模型, 从而提供平衡的准确度与延迟权衡。如果与可达到的最高级联精度相比, 该阈值产生的精度损失超过 1 pp, 则我们使用满足 1 pp 限制的最低阈值。该基线相当于一组最先进的级联 [5]、[6]、[9]。

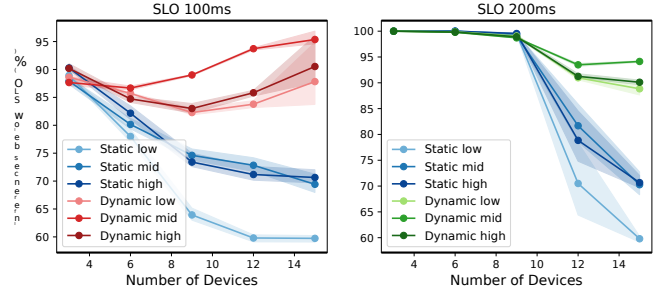


Fig. 7: SLO satisfaction rate for EfficientNetB3.

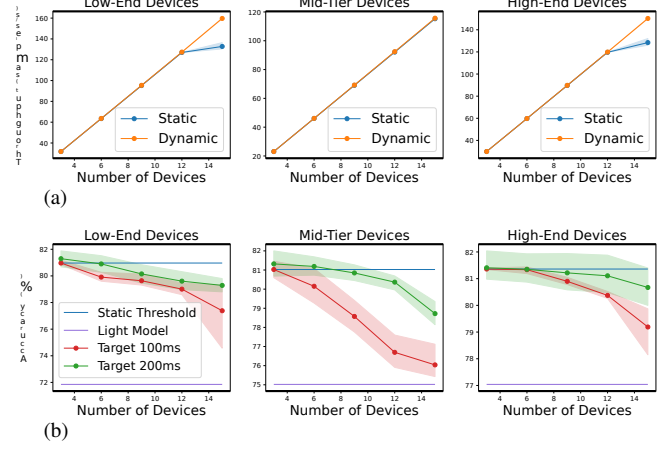


图 8: 服务器上 EfficientNetB3 的 (a) 吞吐量和 (b) 准确性。

B. Evaluation of MultiTASC's Performance

在这里, 我们评估了调度程序在两个设备生态系统中的性能。我们使用网格搜索优化了 MultiTASC 的参数 P 、 M 、 L 、 α 和 β , 并将它们分别设置为 20%、0.05、5、0.83 和 0.125。

同质场景。图 3 显示了 EfficientNetLite0-InceptionV3 对的 SLO 满意度比较。对于少量设备, MultiTASC 使满意度保持接近基线。当基线分别在 100 和 200 毫秒的 SLO 的 20 和 40 台设备上开始失败时, MultiTASC 逐渐重新配置转发决策功能的阈值, 并设法保持较高的满意度, 在 SLO 中获得 15-20 个百分点。在图 4 中, 我们可以看到, 在 45 个设备之后, 由于队列的服务速度不够快, 基线的吞吐量开始趋于稳定。设备的用户会因为响应时间过长而体验到这种停滞状态。相比之下, 使用 MultiTASC, 总吞吐量随着设备数量的增加而继续线性增加, 这表明请求不允许累积并且可以服务更多设备。

在准确性方面, 对于少量设备, MultiTASC 稍微提高了准确性。发生这种情况是因为我们的方法认识到服务器未得到充分利用, 并相应地增加了阈值。随着设备数量的增加, 需要牺牲准确性来实现更低的延迟和维持更高的吞吐量, 但仍保持在基线的 2 pp 以内。此外, 我们观察到级联实现的精度在大多数情况下高于

the accuracy of the heavy model, showcasing the benefits of classifier cascades for the specific model pair. Moreover, we observe the scalability of MultiTASC, serving up to 50 devices without significant loss of accuracy. Different model pairings gave similar results, providing high satisfaction rate, sustaining the throughput and keeping accuracy within acceptable limits. When comparing server-side models, InceptionV3 can serve a larger amount of devices compared to EfficientNetB3, due to its lower computational demands, but EfficientNetB3 achieves higher accuracy when it comes to a lower number of devices.

We further observe that when the on-device inference latency is below the SLO, the lowest limit for the SLO satisfaction rate is the percentage of samples that are not forwarded to the server. For the EfficientNetLite0-InceptionV3 pair, this is around 69% and can be observed when the baseline solution reaches 40 devices for the 100-ms SLO (Fig. 3)

Heterogeneous Scenario. Fig. 5-6 show the SLO satisfaction rate, throughput and accuracy comparison between MultiTASC and the baseline, when targeting a heterogeneous device environment with InceptionV3 as the server-side model. We report the performance results separately for each device tier. When calculating the throughput, different latency targets give similar results, so we present only one. Similarly to the homogeneous case, we observe that the satisfaction rates are maintained high across all tiers of devices, while the baseline leads to catastrophically degraded latency after 30 and 40 devices for SLOs of 100 and 200 ms, respectively (Fig. 5). Our approach achieves gains of approximately 20-25 pp in satisfaction rate across all device tiers. We also observe the throughput plateau, but only for the low- and high-end devices. This is attributed to the fact that the latency of EfficientNetLite0 on the mid-tier devices is $1.32\times$ slower than the other tiers, resulting in mid-tier devices continuing inference after the inference on low-end and high-end devices has finished. This allows the server to decongest since only $1/3$ of the devices remains. Accuracy is again within acceptable values, never dropping below 2 pp from the baseline.

Fig. 7-8 show the performance comparison when EfficientNetB3 is used as the server-side model. Our observations are similar to the previous case of InceptionV3, showcasing the generality of our approach. From this set of experiments, we infer that the highest-performing server-side model is in most cases the higher-throughput one, *e.g.* InceptionV3. Specifically, we observe that InceptionV3 can serve a much larger number of devices while keeping accuracy within acceptable levels compared to the complex, low-throughput EfficientNetB3. EfficientNetB3 achieves higher accuracy when the number of devices is 9 or lower for SLO of 100 ms or 14 and lower for SLO of 200 ms. In all cases, via adjusting the forwarding decision thresholds during execution, MultiTASC manages to keep response times low and system throughput high by gradually trading off accuracy.

VI. CONCLUSION

This paper presents MultiTASC, a novel scheduler that enables cascade collaborative systems when assisting multi-

ple devices at the same time. By dynamically updating the thresholds of forwarding decision functions on devices during run time, MultiTASC manages to avoid request accumulation on server-side which keeps response latency below SLO targets and throughput high across different device pool sizes. Moreover, it considers the device heterogeneity problem and is able to maintain performance while accommodating different tiers of devices. The flexibility and scalability of MultiTASC will help push collaborative cascade systems to enable broader DL deployment in indoor intelligent environments.

REFERENCES

- [1] S. I. Venieris, I. Panopoulos, and I. S. Venieris, "OODIn: An Optimised On-Device Inference Framework for Heterogeneous Mobile Devices," in *SMARTCOMP*, 2021.
- [2] S. Laskaridis, S. I. Venieris, A. Kouris, R. Li, and N. D. Lane, "The Future of Consumer Edge-AI Computing," *arXiv*, 2022.
- [3] M. Almeida, S. Laskaridis, A. Mehrotra, L. Dudziak, I. Leontiadis, and N. D. Lane, "Smart at What Cost? Characterising Mobile Deep Neural Networks in the Wild," in *IMC*, 2021.
- [4] E. Park, D. Kim, S. Kim, Y.-D. Kim, G. Kim, S. Yoon, and S. Yoo, "Big/little deep neural network for ultra low power inference," in *CODES+ISSS*, 2015.
- [5] M. Li, Y. Li, Y. Tian, L. Jiang, and Q. Xu, "AppealNet: An Efficient and Highly-Accurate Edge/Cloud Collaborative Architecture for DNN Inference," in *DAC*, 2021.
- [6] X. Wang, Y. Luo, D. Crankshaw, A. Tumanov, F. Yu, and J. E. Gonzalez, "IDK Cascades: Fast Deep Learning by Learning not to Overthink," in *UAI*, 2018.
- [7] S. I. Mirzadeh and H. Ghasemzadeh, "Optimal Policy for Deployment of Machine Learning Models on Energy-Bounded Systems," in *IJCAI*, 2020.
- [8] D. Stamoulis, T.-W. R. Chin, A. K. Prakash, H. Fang, S. Sajja, M. Bogner, and D. Marculescu, "Designing Adaptive Neural Networks for Energy-Constrained Image Classification," in *ICCAD*, 2018.
- [9] A. Kouris, S. I. Venieris, and C.-S. Bouganis, "CascadeCNN: Pushing the Performance Limits of Quantisation in Convolutional Neural Networks," in *FPL*, 2018.
- [10] T. Nakamura, S. Saito, K. Fujimoto, M. Kaneko, and A. Shiraga, "Spatial- and Time-Division Multiplexing in CNN Accelerator," *Parallel Computing*, 2022.
- [11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *CVPR*, 2018.
- [12] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," in *ICLR*, 2015.
- [13] Y. He, X. Zhang, and J. Sun, "Channel Pruning for Accelerating Very Deep Neural Networks," in *ICCV*, 2017.
- [14] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," in *NeurIPS*, 2014.
- [15] B. Cox, R. Birke, and L. Y. Chen, "Memory-aware and Context-aware Multi-DNN Inference on the Edge," *Pervasive and Mobile Computing*, 2022.
- [16] M. Almeida, S. Laskaridis, S. I. Venieris, I. Leontiadis, and N. D. Lane, "DynO: Dynamic Onloading of Deep Neural Networks from Cloud to Device," *TECS*, 2022.
- [17] J. Huang, C. Samplawski, D. Ganesan, B. Marlin, and H. Kwon, "CLIO: Enabling Automatic Compilation of Deep Learning Pipelines across IoT and Cloud," in *MobiCom*, 2020.
- [18] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge," in *ASPLOS*, 2017.
- [19] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *ICML*, 2017.
- [20] A. J. Joshi, F. Porikli, and N. Papanikolopoulos, "Multi-Class Active Learning for Image Classification," in *CVPR*, 2009.
- [21] A. Ali, R. Pincioli, F. Yan, and E. Smirni, "BATCH: Machine Learning Inference Serving on Serverless Platforms with Adaptive Batching," in *SC*, 2020.

重模型的准确性，展示了分类器级联对于特定模型对的好处。此外，我们观察到 MultiTASC 的可扩展性，可以为多达 50 个设备提供服务，而不会显著损失准确性。不同的模型配对给出了相似的结果，提供了高满意度，维持了吞吐量并将准确性保持在可接受的范围内。在比较服务器端模型时，与 EfficientNetB3 相比，InceptionV3 由于计算需求较低，可以服务更多数量的设备，但 EfficientNetB3 在设备数量较少时实现了更高的准确率。

我们进一步观察到，当设备上的推理延迟低于 SLO 时，SLO 满足率的最低限制是未转发到服务器的样本的百分比。对于 EfficientNetLite0-InceptionV3 对，该比例约为 69%，并且可以在基准解决方案达到 40 个设备且 100 毫秒 SLO 时观察到（图 3）

异构场景。图 5-6 显示了以 InceptionV3 作为服务器端模型的异构设备环境下 MultiTASC 与基线之间的 SLO 满意度、吞吐量和准确率比较。我们单独报告每个设备层的性能结果。在计算吞吐量时，不同的延迟目标会得出相似的结果，因此我们仅提供一种。与同质情况类似，我们观察到所有级别的设备的满意度都保持较高水平，而基线分别导致 30 和 40 台设备的 SLO 为 100 和 200 毫秒时延迟发生灾难性降低（图 5）。我们的方法使所有设备层的满意度提高了大约 20-25 个百分点。我们还观察到吞吐量平台，但仅限于低端和高端设备。这是由于中端设备上 EfficientNetLite0 的延迟比其他层慢 1.32 $\times$ ，导致中端设备在低端和高端设备上的推理完成后继续推理。这允许服务器缓解拥塞，因为仅保留 1/3 台设备。准确度再次处于可接受的值内，从未低于基线 2 pp。

图7-8所示为Efficient-NetB3作为服务器端模型时的性能对比。我们的观察结果与之前的 InceptionV3 案例类似，展示了我们方法的通用性。从这组实验中，我们推断出性能最高的服务器端模型在大多数情况下是吞吐量较高的模型，即 e.g. InceptionV3。具体来说，我们观察到与复杂、低吞吐量的 EfficientNetB3 相比，InceptionV3 可以服务更多数量的设备，同时将准确性保持在可接受的水平内。当设备数量为 9 个或更少（对于 100 ms 的 SLO）或 14 个或更少（对于 200 ms 的 SLO）时，EfficientNetB3 可以实现更高的精度。在所有情况下，通过在执行期间调整转发决策阈值，MultiTASC 通过逐渐权衡准确性来设法保持较低的响应时间和较高的系统吞吐量。

六. 结论

本文介绍了 MultiTASC，这是一种新颖的调度程序，可在协助多任务时启用级联协作系统。

同时使用多个设备。通过在运行时动态更新设备上转发决策函数的阈值，MultiTASC 能够避免服务器端的请求累积，从而使响应延迟低于 SLO 目标，并在不同设备池大小之间保持较高的吞吐量。此外，它考虑了设备异构性问题，能够在适应不同层级设备的同时保持性能。MultiTASC 的灵活性和可扩展性将有助于推动协作级联系统，从而在室内智能环境中实现更广泛的深度学习部署。

参考

- [1] S. I. Venieris, I. Panopoulos 和 I. S. Venieris, “OODIn: 针对异构移动设备的优化的设备端推理框架”, *SMARTCOMP*, 2021 年。
- [2] S. Laskaridis, S. I. Venieris, A. Kouris, R. Li 和 N. D. Lane, “消费者边缘 AI 计算的未来”, *arXiv*, 2022 年。
- [3] M. Almeida, S. Laskaridis, A. Mehrotra, L. Dudziak, I. Leontiadis 和 N. D. Lane, “智能成本如何? 表征野外移动深度神经网络”, *IMC*, 2021 年。
- [4] E. Park, D. Kim, S. Kim, Y.-D. Kim, G. Kim, S. Yoon 和 S. Yoo, “用于超低功耗推理的大/小深度神经网络”, *CODES+ISSS*, 2015 年。
- [5] M. Li, Y. Li, Y. Tian, L. Jiang 和 Q. Xu, “AppealNet: 一种高效、高精度的 DNN 推理边缘/云协作架构”, *DAC*, 2021 年。
- [6] X. Wang, Y. Luo, D. Crankshaw, A. Tumanov, F. Yu, 和 J. E. Gonzalez, “IDK Cascades: 通过学习不要过度思考来实现快速深度学习”, *UAI*, 2018。
- [7] S. I. Mirzadeh 和 H. Ghasemzadeh, “在能量有限系统上部署机器学习模型的最佳策略”, *IJCAI*, 2020。
- [8] D. Stamoulis, T.-W. R. Chin, A. K. Prakash, H. Fang, S. Sajja, M. Bognar 和 D. Marculescu, “设计用于能量受限图像分类的自适应神经网络”, *ICCAD*, 2018 年。
- [9] A. Kouris, S. I. Venieris 和 C.-S. Bouganis, “CascadeCNN: 突破卷积神经网络中量化的性能极限”, *FPL*, 2018 年。
- [10] T. Nakamura, S. Saito, K. Fujimoto, M. Kaneko 和 A. Shiraga, “CNN 加速器中的空间和时分复用”, *Parallel Computing*, 2022 年。
- [11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov 和 L.-C. Chen, “MobileNetV2: 倒置残差和线性瓶颈”, *CVPR*, 2018 年。
- [12] S. Han, H. Mao 和 W. J. Dally, “深度压缩: 通过剪枝、训练量化和霍夫曼编码压缩深度神经网络”, *ICLR*, 2015 年。
- [13] Y. He, X. Zhu 和 J. Sun, “加速极深神经网络的通道修剪”, *ICCV*, 2017。
- [14] G. Hinton, O. Vinyals 和 J. Dean, “在神经网络中提取知识”, *NeurIPS*, 2014。
- [15] B. Cox, R. Birke 和 L. Y. Chen, “内存感知和上下文感知边缘上的多 DNN 推理”, *Pervasive and Mobile Computing*, 2022 年。
- [16] M. Almeida, S. Laskaridis, S. I. Venieris, I. Leontiadis 和 N. D. Lane, “DynO: 从云到设备的深度神经网络动态加载”, *TECS*, 2022 年。
- [17] J. Huang, C. Samplawski, D. Ganesan, B. Marlin 和 H. Kwon, “CLIO: 实现跨物联网和云的深度学习管道自动编译”, *MobiCom*, 2020 年。
- [18] Y. Kang, J. Hauswald, C. 高, A. Rovinski, T. Mudge, J. Mars 和 L. Tang, “神经外科医生: 云和移动边缘之间的协作智能”, *ASPLOS*, 2017 年。
- [19] C. Guo, G. Pleiss, Y. Sun 和 K. Q. Weinberger, “论现代神经网络的校准”, *ICML*, 2017 年。
- [20] A. J. Joshi, F. Porikli 和 N. Papanikolopoulos, “多类主动学习图像分类”, *CVPR*, 2009 年。
- [21] A. Ali, R. Pincirol i, F. Yan 和 E. Smirni, “BATCH: 通过自适应批处理在无服务器平台上提供机器学习推理服务”, *SC*, 2020 年。